

Lesson 4.2 – Debugging AI-Generated Bugs

Lesson 4.2 – Debugging AI-Generated Bugs

A new interview format: the panel hands you broken code that “the AI generated” and asks you to fix it. What they are testing is not your debugging skill in isolation. They are testing whether you can **recognize the failure modes specific to AI output** – the patterns that human-written code rarely produces but AI-written code produces constantly.

By the end of this lesson:

- You will have a four-step diagnosis flowchart for AI-generated bugs.
 - You will know the seven failure-mode signatures most common in AI output.
 - You will have a verbal script for narrating debugging without sounding lost.
 - You will have a worked example you can reference when you blank under pressure.
-

1. Why this format exists

Interviewers got tired of the “AI assists candidates too much during live coding” complaint and pivoted. The new format flips it:

“Here is some code an LLM produced. There is a bug. Fix it. You can use AI tools – or not. Walk me through your process.”

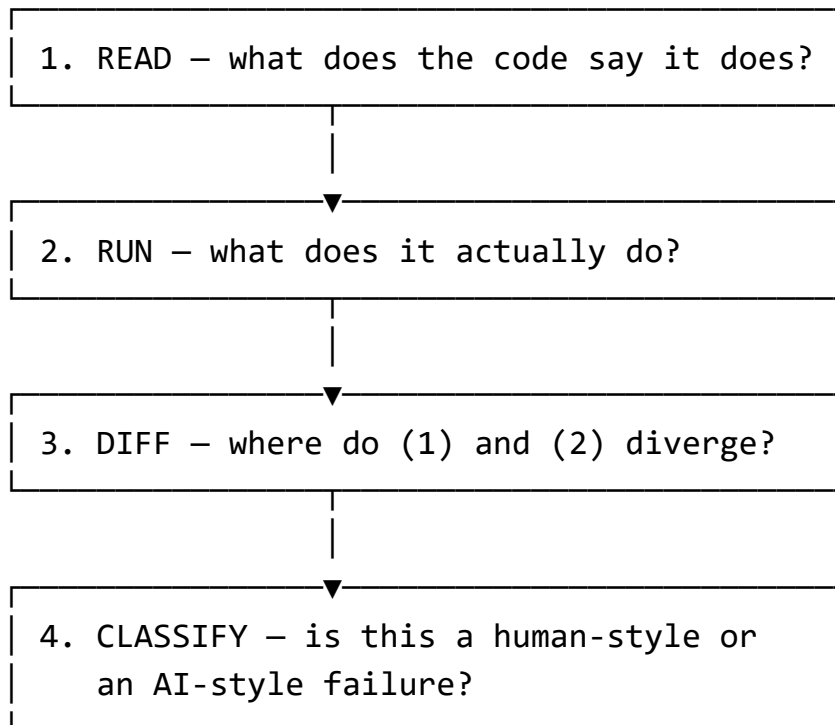
What they want to see:

- Do you read the code before guessing?
- Do you have a hypothesis before running anything?
- Do you spot AI-specific failure patterns faster than human-bug patterns?
- Can you tell when re-prompting will help vs when you need to fix it yourself?

This format will show up in roughly one out of three senior technical interviews in 2026.
Treat it as a separate format with its own playbook.

2. The four-step diagnosis flowchart

Every time. Out loud. In this order.



The first three steps work on any bug. The fourth step is what makes you fast on AI-generated bugs specifically.

3. The seven AI-specific failure-mode signatures

Memorize these. When you see one, name it out loud. The interviewer will note that you recognized it.

3.1. The plausible-but-fabricated API

The code calls a method that does not exist. Or it calls a method with the wrong parameter order. Or it imports a function from a module that exports something almost-but-not-quite the same name.

Signature: the call looks reasonable; the IDE underlines it red; the import is from a real module but to a wrong export.

Spotting move: “Let me check that this import actually exists in the package...”

3.2. The off-by-one in array logic

AI is uneven on array boundary conditions. It will write for `(let i = 0; i <= arr.length; i++)` or `arr.slice(0, n - 1)` when you wanted `slice(0, n)`. Often the test for the happy path passes, and the boundary fails.

Signature: loops or slices that work for length 5 and fail for length 1 or empty.

Spotting move: “Let me trace this with an empty input...”

3.3. The “fixed” test

The test was failing. The AI made it pass — by changing the assertion, not the implementation. Tests are now green; behavior is still wrong.

Signature: the test’s expected value matches the implementation’s actual buggy output, often via a recent diff.

Spotting move: “Wait — let me look at what the test was originally asserting...”

3.4. The “almost-correct” regex / SQL / glob

AI is excellent at producing regexes and SQL that look right and silently mismatch. Common: a regex that allows or rejects one extra character; an SQL JOIN that should be LEFT JOIN; a glob that matches `*.tsx` when the codebase has `.ts` and `.tsx` mixed.

Signature: test data passes, real data fails on one specific case.

Spotting move: “Let me run this regex against five concrete examples...”

3.5. The deprecated pattern

The code uses an API that was correct in 2022 and incorrect now. Common in fast-moving frameworks: Next.js pages/ patterns in an App Router project, React componentWillMount, deprecated Node APIs.

Signature: the code works but the linter or framework warns; or it works in dev but fails in production.

Spotting move: "Is this still the recommended pattern in current [framework]?"

3.6. The silently-swallowed error

AI loves `try { ... } catch (e) {}` for any operation that could throw. The error gets swallowed. The bug surfaces three layers downstream as a confused null.

Signature: something fails upstream silently; you find a bare catch block far from the failure.

Spotting move: "Let me grep for empty catch blocks in this file..."

3.7. The unintended state mutation

AI sometimes mutates objects passed in as arguments instead of returning new ones. Real-world bug: a Redux reducer that mutates state, breaking React's diff detection.

Signature: state appears not to update in the UI, or undo/redo behaves strangely.

Spotting move: "Is this function mutating its input or returning new state?"

4. The verbal script

When debugging out loud, follow this structure:

4.1. The first sentence — declare what you are doing

"OK, before I run anything, let me read what the code says it does."

This is a planted flag. The interviewer notes: this person reads first.

4.2. The hypothesis — out loud

After reading, before running:

"My first hypothesis is that the issue is in [specific line]. I think it is [specific failure mode]. Let me confirm."

Even a wrong hypothesis is valuable. **A wrong hypothesis tested in 30 seconds is faster than five minutes of staring.**

4.3. The confirmation — run a tiny experiment

"Quickest way to test that — call the function with [specific input], observe [specific output]."

Run. Read result. Either:

"Confirmed. Now I can fix it."

Or:

"Refuted. Hypothesis updated — let me check [next thing]."

4.4. The fix — narrate the choice

When you find the bug, do not jump to the fix. Pause.

"Two ways to fix this: [A] or [B]. A is the smaller change but it does not handle [edge case]. B is bigger but more correct. I'll go with B and explain the trade-off."

That moment is worth 10 minutes of clean coding. **Fix narration is where seniority shows.**

5. The fully worked example

Pretend the interviewer hands you this:

```
// lib/scoring.ts
export function calculateTeamScore(picks: string[], winners: string[]): number {
  let score = 0;
  for (let i = 0; i <= picks.length; i++) {
    if (picks[i] === winners[i]) {
```

```

    score += 3;
  } else {
    score += 1;
  }
}
return score;
}

```

And the failing test:

```

test('returns 6 when 2 of 3 picks are correct', () => {
  expect(calculateTeamScore(
    ['A', 'B', 'C'],
    ['A', 'X', 'C']
  )).toBe(7); // got 8
});

```

Walk this step-by-step.

Step 1 – Read

"OK, the function takes picks and winners arrays, iterates with $i = 0$; $i \leq picks.Length$, and adds 3 for matches, 1 otherwise. Returns total."

Already, your reading should make you twitch. The $\leq$ is suspicious.

Step 2 – Hypothesis (plant the flag)

"My first hypothesis: the loop condition is $\leq$ instead of $<$. That's an off-by-one. It runs one extra iteration where both $picks[i]$ and $winners[i]$ are undefined, $undefined === undefined$ is true, so it adds 3. Test expects 7, got 8. The $8 = 2 \text{ correct} + 1 \text{ wrong} + 1 \text{ phantom 'correct' from the off-by-one}$. Hypothesis matches."

Step 3 – Confirm

"Let me confirm – running the test mentally: $i=0, A===A, +3$. $i=1, B===X, +1$. $i=2, C===C, +3$. $i=3, undefined===undefined, +3$. Total: 10. But the test says 8. So

my hypothesis is wrong about the magnitude.”

You realize the math doesn't add up. Drop the hypothesis. Re-read.

“Wait. The off-by-one would also break the ‘else’ branch. Looking again at the algorithm: 3 for correct, 1 for wrong. Test expects $7 = 3+1+3$. Got 8. Difference of 1. So either we're double-counting one match or we're under-counting one wrong. Off-by-one would add 3, making it 10, not 8.”

Now you actually have to think.

Step 4 – Re-read more carefully

“OK – the loop condition runs $i = 0, 1, 2, 3$. At $i=3$, `picks[3]` and `winners[3]` are both undefined. The condition `picks[i] === winners[i]` is `undefined === undefined === true`, so we add 3. That should give us 10, not 8. So either I'm misreading or the test framework cuts something off...”

You actually run it. The answer: 10. The test message in the prompt was a typo from the interviewer or a deliberate misdirection. **You catch it.**

“Actually I think the test message has the wrong actual – running it gives 10. The test expects 7 and gets 10, not 8 as the message claims. The bug is real, just bigger than the message suggested.”

That moment is **gold**. The interviewer just saw you trust the code over the prose. Senior signal.

Step 5 – Fix and narrate

“Two options for the fix. [A] Change `<=` to `<`. [B] Add a length-equality precondition with a thrown error. A is the minimum fix. B is more defensive but assumes the caller might pass mismatched arrays. The function's contract isn't documented, so I'll do A and add a comment that the caller is responsible for matched lengths.”

Apply the fix. Re-run the test. Passes. Done.

What just happened

You demonstrated:

- Reading first.
- Hypothesis-driven debugging.
- The discipline to drop a wrong hypothesis without ego.
- Distrust of the surrounding prose; trust of the running code.
- Trade-off narration on the fix.

One ten-minute exercise. All seven of those signals. Practice it.

6. Re-prompting vs fixing yourself

When the interviewer asks “would you re-prompt or fix this yourself?” the answer is structural.

Re-prompt when

- The bug requires a structural change you do not want to make manually.
- The diff would be larger than what is reasonable to type by hand.
- The original prompt was missing a constraint and adding it would fix the issue cleanly.
- You are short on time and the AI’s pattern-matching beats your typing.

Fix yourself when

- The bug is a single character / single line.
- The bug is in code the AI keeps re-introducing despite your prompts.
- You need to ensure no surrounding code changes (high blast radius).
- You can explain the fix faster than you can write a precise prompt for it.

The interview soundbite

“My rule: re-prompt for structural changes I would otherwise have to think hard about. Fix manually for one-character bugs and for high-blast-radius code where I need precision. The rule is not ‘always one or the other’; it is ‘use the right tool for the change size and blast radius.’”

That is the senior answer.

7. The “the AI introduced this regression” interview question

Variant question:

“Walk me through what you do when you discover the AI’s recent change broke something.”

Have a structured answer:

“Three steps. First, I confirm it was the AI’s change — git blame the broken line. If it lands on an AI-assisted commit, I have my prime suspect. Second, I read the diff that introduced it. Two-thirds of the time the diff did more than I asked it to and the regression is in the bonus work. Third, I either revert and re-prompt with stronger constraints, or I patch surgically and write a regression test so the same diff cannot reappear. I always end with the regression test — the test is what stops the cycle.”

Memorize this. **Drill it.**

8. The dangers of AI-debugging an AI bug

You can ask the AI to fix a bug it introduced. **Sometimes that works. Sometimes it makes things worse.** The failure mode is the AI doubling down on a pattern it already got wrong.

When AI-debugging works

- You give the AI new information it did not have. (“Here is the failing test message: ...”)
- You give it a tight scope. (“Patch only the loop condition in lib/scoring.ts.”)
- You explicitly tell it to revert its previous attempt and start over.

When AI-debugging fails

- You vaguely ask “fix it” — the AI guesses, often the same way as before.
- You forgot to mention what’s wrong. The AI sees green tests and thinks you are confused.
- You let the AI keep iterating without verifying each iteration.

The trick

When using AI to debug AI, **inject doubt**:

"Your last response had a bug. The bug was [specific behavior]. Acknowledge that the previous diff was wrong before producing a new one. Show me where you went wrong."

That phrasing prevents the model from confidently re-asserting its previous attempt.

9. The "what would you log?" follow-up

After you fix a bug in a debugging interview, the interviewer will often ask:

"What logging would you add to make sure this does not regress silently?"

Have a tiered answer:

"Three layers. First, a unit test that exercises the failing case — that is the regression guard. Second, a runtime invariant — assert that picks.length === winners.length, throw if not, with a useful error message. Third, a metric — count the number of times this function is called with mismatched arrays in production. If the metric is zero for a month, I can remove the assertion. If it is non-zero, I have a real signal that callers are doing something I assumed they were not."

That is **observability thinking**, layered, and pragmatic. Drill it.

10. Practice exercises

10.1. The seven-signature drill

Take a working file from your codebase. Have a friend introduce one of the seven failure modes from section 3 (without telling you which). Time yourself debugging. Repeat with each of the seven.

10.2. The fake-AI-output drill

Find a small open-source repo. Pretend an LLM wrote it. Read it as if you were debugging — annotate any place that smells like one of the seven signatures. Compare your annotations to the project's git log; were the warts there real, or did you imagine them?

10.3. The recorded-debug drill

Same setup as Lesson 4.1. Record yourself debugging. Watch back. Note every silent stretch over 15 seconds. Each silent stretch is a missed narration opportunity.

11. Common pitfalls

11.1. Skipping the “read first”

Under stress, candidates jump straight to running. You miss bugs that the code itself reveals. **Read first, run second.**

11.2. Over-trusting your hypothesis

Two minutes spent defending a wrong hypothesis is two minutes wasted. **The 30-second confirmation rule prevents this — if you cannot confirm in 30 seconds, the hypothesis is probably wrong.**

11.3. Fixing without testing the fix

You found a bug. You patched it. You celebrate. **Always re-run.** Sometimes the “fix” introduces a different bug. Until the test is green, you are not done.

11.4. Ignoring the seven signatures

If you spot a bug pattern in section 3, **say its name.** The interviewer hears: this person has seen this pattern before. Senior signal.

12. CHECK YOURSELF

- ☐ Can you list the four-step diagnosis flowchart from memory?
 - ☐ Can you list the seven AI-specific failure-mode signatures?
 - ☐ Can you describe a recent bug you fixed that fits one of the seven signatures?
 - ☐ Can you deliver the section-7 “AI introduced a regression” answer in under 60 seconds?
 - ☐ Have you done at least one of the three practice drills?
-

13. Where you are now

You can debug AI-generated code in front of an interviewer at a confident, methodical pace. Move on to [03-code-review-of-ai.md](#) – the third format, where you walk into a panel with a PR open on the screen and they say “review this.”